Relatório de Auditoria Técnica — Conformidade CRUD

Projeto Integrador IV - B · Sistema de Consulta e Gestão de Pontos de Vacinação em Alagoas

Metadado	Detalhe
Documento	`AUDITORIA-CRUD-PI4B`
Data da Auditoria	01/11/2026
Escopo Analisado	Backend FastAPI (`backend/src`), Banco PostgreSQL (`sql/`), Frontend Web (`frontend/public`) e Testes (`backend/tests`)
Status Geral	**Atendimento Parcial (Create & Read implementados; Update & Delete ausentes)**

1. Resumo Executivo

A auditoria avaliou se o sistema atual cumpre integralmente as quatro operações fundamentais de persistência — **C**reate (Criação), **R**ead (Leitura), **U**pdate (Atualização) e **D**elete (Exclusão) — tanto na camada de API RESTful quanto na interface administrativa.

Conclusão:

- O sistema **atende 100% dos requisitos funcionais originais do MVP (RF-01 a RF-05)** descritos no PRD, que previam apenas listagem pública (`GET`) e inserção administrativa (`POST`).
- Contudo, sob a ótica de um **CRUD completo**, o sistema **atende parcialmente**: possui as operações **Create** e **Read**, mas **não possui endpoints nem regras de serviço para Update (`PUT` / `PATCH`) e Delete (`DELETE`)**.
- Além disso, foram identificadas **divergências de integração no painel administrativo (`frontend/public/admin.html`)**, onde botões de edição/exclusão e chamadas `fetch` estão desconectados dos IDs do DOM e das rotas reais do backend.

2. Matriz de Conformidade CRUD por Entidade

Entidade / Tabela	**C**reate (`POST`)	**R**ead (`GET`)	**U**pdate (`PUT` / `PATCH`)	**D**elete (`DELETE`)	Diagnóstico Técnico
Pontos de Vacinação (`vaccination_points`)	✅ Implementado	✅ Implementado	❌ **Ausente**	❌ **Ausente**	Possui criação individual e em lote (OSM) e leitura por cidade. Não possui leitura unitária por ID do posto, atualização cadastral ou exclusão.
Municípios (`cities`)	❌ Ausente	✅ Implementado	❌ Ausente	❌ Ausente	Possui listagem geral (`GET /cities`) e busca por ID (`GET /cities/{id}`). O schema `CityCreate` existe em `schemas/city.py` , mas sem rota ou método no `CityService` .
Vacinas / Imunizantes (`vaccines`)	❌ Ausente	⚠️ Indireto	❌ Ausente	❌ Ausente	As vacinas são retornadas agregadas dentro dos postos (`ARRAY_AGG`), mas não há endpoint dedicado (`GET /vaccines`) nem CRUD próprio para gerir o catálogo.
Estados (`states`)	❌ Ausente	⚠️ Indireto	❌ Ausente	❌ Ausente	Carregado via `sql/seed.sql` e exibido nos detalhes do município (`GET /cities/{id}`).
Administradores (`admin_users`)	❌ Ausente	❌ Ausente	❌ Ausente	❌ Ausente	Usuário inicial criado no `seed.sql` e autenticado via `POST /auth/login` .

3. Análise Detalhada da Camada Backend (`backend/src`)

3.1. Operações Implementadas e Funcionais (Create & Read)

1. **Read — Municípios (`backend/src/api/v1/endpoints/cities.py`):**
    - `GET /cities` (e `/api/v1/cities`): Lista todos os municípios ordenados por ID (`CityService.list_cities`).
    - `GET /cities/{id}` (e `/api/v1/cities/{id}`): Retorna detalhes do município e nome do estado com tratamento de erro `404 Not Found` e `422 Unprocessable Entity` (`CityService.get_city_by_id`).
2. **Read — Pontos de Vacinação (`backend/src/api/v1/endpoints/points.py`):**
    - `GET /vaccination-points/{city_id}` (e `/api/v1/vaccination-points/{city_id}`): Valida a existência do município e executa query SQL otimizada com `LEFT JOIN` e `ARRAY_AGG` para retornar os postos com suas respectivas vacinas (`PointService.list_by_city`).
3. **Create — Pontos de Vacinação (`backend/src/api/v1/endpoints/points.py` & `enrichment.py`):**
    - `POST /vaccination-points` (protegido por JWT via `Depends(get_current_admin)`): Insere um novo posto na tabela `vaccination_points` e vincula os imunizantes na tabela associativa `point_vaccines` (`PointService.create_point`).
    - `POST /admin/enrichment/confirm-save` (protegido por JWT): Insere múltiplos postos minerados via OpenStreetMap de forma idempotente (`ON CONFLICT (city_id, name) DO NOTHING`).

3.2. Lacunas para o CRUD Completo no Backend (Update & Delete)

1. **Ausência de Update (`PUT /vaccination-points/{point_id}` ou `PATCH`):**
    - **Schema (`backend/src/schemas/point.py`):** Existe apenas `VaccinationPointCreate` e `VaccinationPointResponse` . Falta o schema `VaccinationPointUpdate` com campos opcionais para edição parcial ou total.
    - **Serviço (`backend/src/core/services/point_service.py`):** Falta o método `update_point(point_id, payload)` que localize o registro existente, atualize atributos (`name` , `neighborhood` , `address` , `latitude` , `longitude` , `city_id`), sincronize os vínculos na tabela `point_vaccines` e faça `commit` .
    - **Endpoint (`backend/src/api/v1/endpoints/points.py`):** Falta a rota `@router.put("/{point_id}")` protegida por JWT.
2. **Ausência de Delete (`DELETE /vaccination-points/{point_id}`):**

- **Banco de Dados (`sql/init.sql` e `entities.py`):** O banco já está devidamente preparado com `ON DELETE CASCADE` na tabela associativa `point_vaccines`, garantindo integridade referencial ao apagar um posto.
- **Serviço (`backend/src/core/services/point_service.py`):** Falta o método `delete_point(point_id)` que verifique se o posto existe (lançando `404 Not Found` caso contrário) e execute a remoção.
- **Endpoint (`backend/src/api/v1/endpoints/points.py`):** Falta a rota `@router.delete("/{point_id}", status_code=204)` protegida por JWT.

3. **Ausência de Leitura Unitária de Posto (`GET /vaccination-points/detail/{point_id}`):**
    - Atualmente, o parâmetro `{city_id}` ocupa a rota `GET /vaccination-points/{city_id}`. Para facilitar o preenchimento de modais de edição por ID único, recomenda-se uma rota ou suporte de busca unitária de posto.

4. Auditoria de Integração no Painel Administrativo (`frontend/public/admin.html`)

A inspeção do código-fonte de `frontend/public/admin.html` revelou que a interface visual possui elementos desenhados para o CRUD, mas apresenta **4 inconsistências técnicas** que impedem seu funcionamento real ponta a ponta:

4.1. Botões de Edição e Exclusão são Estáticos (Mock)

- Nas linhas `602–607` (e demais linhas estáticas da tabela de auditoria), os botões de ação na coluna **"Ações de Gestão"** não realizam chamadas HTTP:
    - O botão **Editar Unidade** (`<span class="material-symbols-outlined">edit</span>`) não possui atributo `onclick` nem função associada.
    - O botão **Desativar/Excluir Unidade** (`<span class="material-symbols-outlined">block</span>`) executa apenas um `alert(...)` estático no navegador, sem enviar requisição `DELETE` ao backend.
- Na renderização dinâmica via JavaScript (`window.carregarPostosAdmin`, linhas `1072–1097`), a última coluna renderiza apenas o ícone do Google Maps (`location_on`), omitindo os botões de **Editar** e **Excluir** para os postos vindos da API.

4.2. Divergência na Rota de Criação Manual (`POST`)

- Na função `window.salvarPontoManual` (linha `1015`), a requisição é disparada para:

```
javascript
fetch(`${API_URL}/admin/vaccination-points`, { method: "POST", ... })
```

- Entretanto, em `backend/src/app/main.py` (linha `38`) e `backend/src/api/v1/api.py` (linha `9`), o roteador está montado em:
  - `POST /vaccination-points`
  - `POST /api/v1/vaccination-points`
- Não existe o prefixo `/admin/vaccination-points` no FastAPI, o que gera erro `404 Not Found` ao tentar salvar pelo painel.

4.3. Desconexão de IDs e Eventos no DOM do `admin.html`

O script ao final do arquivo (`linhas 875–1109`) procura por seletores HTML que divergem dos IDs declarados no corpo do documento:

Funcionalidade	ID / Função no HTML (`admin.html`)	ID / Função esperada no `<script>`	Impacto
Submit de Novo Posto	`onsubmit="... window.simularCadastroPosto()"` (linha `401`)	`window.salvarPontoManual()` (linha `985`)	Ao submeter o formulário, ocorre `TypeError: window.simularCadastroPosto is not a function`.
Campos do Formulário	`campo-municipio`, `campo-nome`, `campo-bairro`, `campo-endereco`, `campo-lat`, `campo-lng`, `name="vacinas"` (`linhas 409–491`)	`novo-posto-cidade`, `novo-posto-nome`, `novo-posto-bairro`, `novo-posto-endereco`, `name="vax"` (`linhas 991–1001`)	Os valores digitados pelo usuário não são lidos pela função de salvamento.
Tabela de Auditoria	`<tbody id="tabela-postos-corpo">` (linha `570`)	`document.getElementById("tbody-postos-admin")` (linha `1042`)	`carregarPostosAdmin()` retorna `null` e nunca atualiza a tabela com os dados reais do PostgreSQL.
Botão de Login / Renovar	Sem ID nos inputs de usuário/senha (`linhas 153, 157`) e botão sem `onclick` (linha `161`)	`admin-user-input`, `admin-pass-input`, `btn-login-admin` (`linhas 911–913`)	O formulário de reautenticação rápida não dispara `window.realizarLoginAdmin()`.

4.4. Divergência na Rota do Bot de Mineração OSM

- Na função `window.executarBotOSM` (linha `963`), o script chama `POST ${API_URL}/admin/enrichment/mine/Maceio`.
- As rotas implementadas em `backend/src/api/v1/endpoints/enrichment.py` são:
  - `GET /admin/enrichment/preview-osm`
  - `POST /admin/enrichment/confirm-save`
  - `POST /admin/enrichment/sync-osm`

5. Auditoria da Suíte de Testes (`backend/tests/api/test_endpoints.py`)

Atualmente, a suíte automatizada conta com **6 testes** que validam apenas leitura (`GET`) e falha de login (`POST /auth/login`):

- `test_get_cities_returns_200`
- `test_get_city_by_id_success`
- `test_get_city_by_id_not_found`
- `test_get_city_by_id_invalid`
- `test_get_vaccination_points_by_city`
- `test_auth_login_invalid_credentials`

O que falta testar para cobrir o CRUD:

- Teste de login com sucesso gerando token JWT válido.
- Teste de criação de posto (`POST /vaccination-points`) autenticado (`201 Created`) e não autenticado (`401 Unauthorized`).
- Teste de atualização de posto (`PUT /vaccination-points/{id}`) autenticado (`200 OK`) e para ID inexistente (`404 Not Found`).
- Teste de exclusão de posto (`DELETE /vaccination-points/{id}`) autenticado (`200 OK` / `204 No Content`) e verificação de remoção em cascata.

6. Plano de Ação Recomendado para Atingir 100% de Conformidade CRUD

Para que o projeto atenda integralmente aos requisitos de um CRUD completo no Backend, Banco de Dados, Frontend e Testes, recomenda-se a execução das seguintes etapas:

1. **Backend — Schemas (`backend/src/schemas/point.py`):**
    - Criar o modelo `VaccinationPointUpdate` permitindo atualizar `city_id`, `name`, `neighborhood`, `address`, `latitude`, `longitude` e `vaccine_ids`.
2. **Backend — Service (`backend/src/core/services/point_service.py`):**
    - Implementar `update_point(point_id: int, payload: VaccinationPointUpdate)` atualizando os dados na tabela `vaccination_points` e recriando as associações em `point_vaccines`.
    - Implementar `delete_point(point_id: int)` removendo o registro do banco de dados.
3. **Backend — Endpoints (`backend/src/api/v1/endpoints/points.py`):**
    - Adicionar `PUT /{point_id}` (protegido com JWT) para atualização.
    - Adicionar `DELETE /{point_id}` (protegido com JWT) para exclusão.
    - *(Opcional)* Adicionar alias `/admin/vaccination-points` ou padronizar todas as chamadas em `/vaccination-points`.
4. **Frontend (`frontend/public/admin.html`):**
    - Alinhar os IDs dos inputs e da tabela (`tabela-postos-corpo`, `campo-nome`, etc.) com o JavaScript.
    - Conectar o formulário para criar (`POST`) e editar (`PUT`) postos reais no PostgreSQL.
    - Adicionar botões funcionais de **Editar** (preenchendo o formulário com os dados do posto selecionado) e **Excluir** (`DELETE` com confirmação) em cada linha da tabela de auditoria.
    - Conectar os botões de prévia (`GET /admin/enrichment/preview-osm`) e confirmação (`POST /admin/enrichment/confirm-save`) do Bot OSM.
5. **Testes Automatizados (`backend/tests/api/test_endpoints.py`):**
    - Adicionar testes cobrindo todo o ciclo de vida CRUD (`POST` → `GET` → `PUT` → `DELETE`).